Intro to Deep Learning

Big Data y Machine Learning para Economía Aplicada

Ignacio Sarmiento-Barbieri

Universidad de los Andes

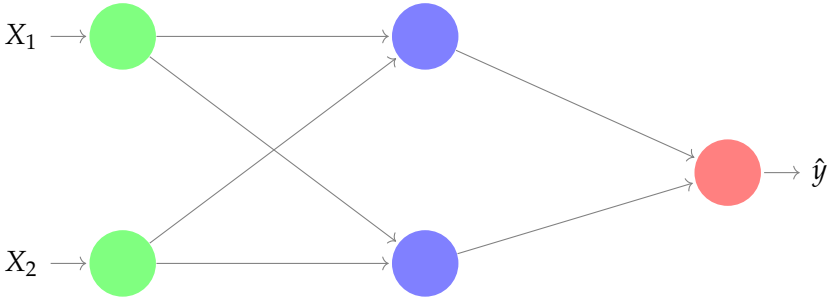
Deep Learning: Intro

- Linear Models may miss the nonlinearities that best approximate $f^*(x)$
- Neural networks are simple models.
- The model has **linear combinations** of inputs that are passed through **nonlinear activation functions** called nodes (or, in reference to the human brain, neurons).

Agenda

1. Recap: SNN
   - Example: XOR
   - Activation Functions
   - Output Functions
2. Training the network
3. Batch, Stochastic, Mini-batch
4. Optimizadores
5. Cierre

Single Layer Neural Networks



Single Layer Neural Networks

- NN are made of **linear combinations** of inputs that are passed through **nonlinear activation functions**
- The NN model has the form

$$f(X) = f\left[\beta_0 + \sum_{k=1}^{K} \beta_k A_k\right] \tag{1}$$

$$= f\left[\beta_0 + \sum_{k=1}^{K} \beta_k g\left(w_{k0} + \sum_{j=1}^{p} w_{kj} X_j\right)\right] \tag{2}$$

- where
  - $g(.)$ is a activiation function, the nonlinearity of $g(.)$ is **key**
  - f is the output layer of the network
- both are prespecified

Worked Example I: Single Layer Neural Networks

- $p = 2$, $X = (X_1, X_2)$
- $K = 2$, $h_1(X)$ and $h_2(X)$
- $g(z) = z^2$

Worked Example I: Single Layer Neural Networks

$$f(X) = \beta_0 + \sum_{k=1}^{2} \beta_k g\left(w_{k0} + \sum_{j=1}^{2} w_{kj} X_j\right) \tag{3}$$

- Suppose we get

$$\begin{array}{ccc} \hat{\beta}_0 = 0 & \hat{\beta}_1 = \frac{1}{4} & \hat{\beta}_2 = -\frac{1}{4} \\ \hat{w}_{10} = 0 & \hat{w}_{11} = 1 & \hat{w}_{12} = 1 \\ \hat{w}_{20} = 0 & \hat{w}_{21} = 1 & \hat{w}_{22} = -1 \end{array}$$

Worked Example I: Single Layer Neural Networks

$$f(X) = \beta_0 + \sum_{k=1}^{2} \beta_k g\left(w_{k0} + \sum_{j=1}^{2} w_{kj} X_j\right) \tag{4}$$

¿Por qué no activaciones lineales?

- Supongamos ahora $g(z) = z$

$$f(X) = \beta_0 + \sum_{k=1}^{2} \beta_k \left(w_{k0} + \sum_{j=1}^{2} w_{kj} X_j \right)$$

- Reemplazando:

$$f(X) = \beta_0 + \beta_1 \left(w_{10} + w_{11} X_1 + w_{12} X_2 \right) + \beta_2 \left(w_{20} + w_{21} X_1 + w_{22} X_2 \right)$$

- Agrupando términos:

$$f(X) = \theta_0 + \theta_1 X_1 + \theta_2 X_2$$

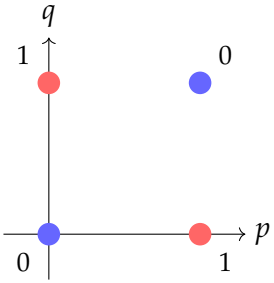
Agenda

1. Recap: SNN
   - Example: XOR
   - Activation Functions
   - Output Functions
2. Training the network
3. Batch, Stochastic, Mini-batch
4. Optimizadores
5. Cierre

Worked Example II: The "Exclusive OR (XOR)" Function

- La disyunción exclusiva de (p, q) significa: "p es verdadero *o* q es verdadero, pero no ambos".
- Tabla de verdad:

q	p	qvp
0	0	0
0	1	1
1	0	1
1	1	0



Rojos: $y = 1$ Azules: $y = 0$

- Cuando uno de los valores toma 1, la funcion XOR retorna 1, en caso contrario, 0.

Worked Example II: The "Exclusive OR (XOR)" Function

- Let's use a linear model

$$y = \beta_0 + \beta_1 q + \beta_2 p + u \tag{5}$$

Worked Example II: The "Exclusive OR (XOR)" Function

- Let's use Single Layer NN containing two hidden units
- Activation Funcition: ReLU: $g(z) = max\{0, z\}$
- NN

$$f(X) = \beta_0 + \sum_{k=1}^{2} \beta_k g\left(w_{k0} + \sum_{j=1}^{2} w_{kj} X_j\right) \tag{6}$$

Worked Example: The "Exclusive OR (XOR)" Function

- Suppose this is the solution to the XOR problem

$$f(x) = max\{0, XW + W_0\}\,\beta + \beta_0$$

$$W = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$$

$$W_0 = \iota_4 \begin{pmatrix} 0 & -1 \end{pmatrix}$$

$$\beta = \begin{pmatrix} 1 & -2 \end{pmatrix}$$

$$\beta_0 = 0$$

Worked Example: The "Exclusive OR (XOR)" Function

Lets work out the example step by step

Agenda

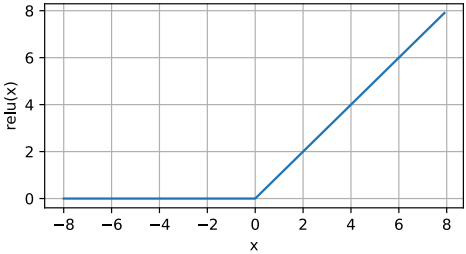
1. Recap: SNN
   - Example: XOR
   - Activation Functions
   - Output Functions
2. Training the network
3. Batch, Stochastic, Mini-batch
4. Optimizadores
5. Cierre

Activation Functions

- Una función de activación $g : \mathbb{R} \to \mathbb{R}$ introduce la *no-linealidad*.
- Propiedades deseables:
  - No-lineal.
  - Diferenciable (necesario para backprop; ReLU es diferenciable salvo en 0).
  - Gradientes "bien comportados".
  - Computacionalmente barata.

Activation Functions

- ReLU function retains only positive elements and discards all negative elements by setting them to 0.
- It is piecewise linear.



Activation Functions

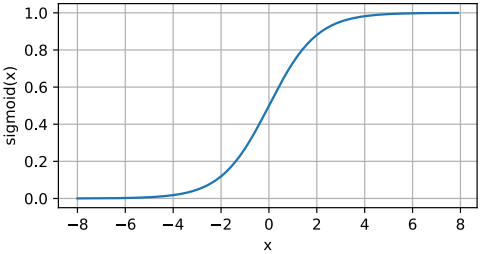
Sigmoid Function (Logit)

- The sigmoid function transforms its inputs, for which values lie in the domain $\mathbb{R}$, to outputs that lie on the interval (0, 1).
- For that reason, the sigmoid is often called a squashing function: it squashes any input in the range (-inf, inf) to some value in the range (0, 1):

$$\text{sigmoid}(x) = \frac{1}{1 + \exp(-x)}.$$

Activation Functions

Sigmoid Function (Logit)



- In the earliest neural networks, scientists were interested in modeling biological neurons which either fire or do not fire. Thus the pioneers of this field, going all the way back to McCulloch and Pitts, the inventors of the artificial neuron, focused on thresholding units.
- A thresholding activation takes value 0 when its input is below some threshold and value 1 when the input exceeds the threshold.
- When attention shifted to gradient based learning, the sigmoid function was a natural choice because it is a smooth, differentiable approximation to a thresholding unit.

Activation Functions

- El diseño de unidades ocultas sigue siendo un área de investigación activa.
- Muchas variantes en uso actual:
  - Leaky ReLU: $g(z) = \max(\alpha z, z)$.
  - ELU, GELU, SiLU / Swish (default en Transformers modernos).
  - Softplus: $\log(1 + e^z)$, aproximación suave de ReLU.
  - Entre otras (ver mas aqui)

Agenda

1. Recap: SNN
   - Example: XOR
   - Activation Functions
   - Output Functions
2. Training the network
3. Batch, Stochastic, Mini-batch
4. Optimizadores
5. Cierre

Output Functions

- The choice of output unit is related to the problem at hand
  - Regression
  - Classification
    - Binary
    - Multiclass
- La output function determina qué pérdida (loss) tiene sentido.

Output y pérdida: el match correcto

Problema	Output $f(\cdot)$	Pérdida L
Regresión	identidad	MSE
Clasif. binaria	sigmoide	cross-entropy
Clasif. multiclase	softmax	cross-entropy

- Cada pérdida sale de la log-verosimilitud del modelo probabilístico correspondiente.
- *No* se mezclan: no tiene sentido usar MSE con softmax, ni cross-entropy con identidad.

Agenda

1. Recap: SNN
   - Example: XOR
   - Activation Functions
   - Output Functions
2. Training the network
3. Batch, Stochastic, Mini-batch
4. Optimizadores
5. Cierre

Training the network

- El objetivo es

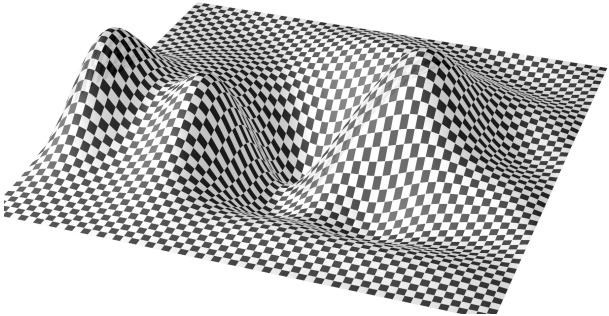
$$\hat{f} = \arg\min_{f} \left\{ \sum_{i=1}^{n} L\big(y_i, f(X_i; \Theta)\big) \right\} \tag{7}$$

- Para una Single Layer NN:

$$f(X, \beta, w) = f\left[\beta_0 + \sum_{k=1}^{K} \beta_k g\left(w_{k0} + \sum_{j=1}^{p} w_{kj} X_j\right)\right] \tag{8}$$

- Los parámetros a estimar son $\Theta = (w, \beta)$.

Training the network



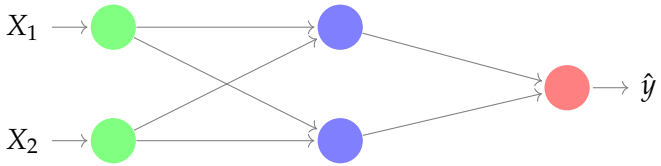
- $L(.)$ es una superficie en un espacio de altas dimensiones.
- Queremos llegar al *mínimo* (o cerca).
- A diferencia de OLS, la superficie ya **no es convexa**: puede haber mínimos locales, puntos silla, valles.
- No hay fórmula cerrada: hay que **bajar paso a paso**.

Training the network

Example: House Prices

Training the network

Example: House Prices



- **Hidden layer** (sigmoide):
  - $A_1 = \sigma(w_{11}X_1 + w_{12}X_2 + w_{10})$
  - $A_2 = \sigma(w_{21}X_1 + w_{22}X_2 + w_{20})$
- **Output** (identidad): $\hat{y}_i = \beta_0 + \beta_1 A_1 + \beta_2 A_2$
- **Pérdida**: MSE $= \frac{1}{n}\sum_i (y_i - \hat{y}_i)^2$

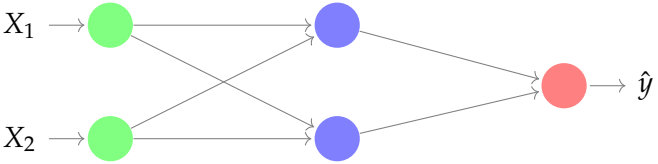
Gradiente descendente

$$\theta' = \theta - \epsilon \cdot \nabla_\theta \mathcal{L}(\theta)$$

- $\epsilon > 0$ es la *learning rate*.
- Idea: en cada iteración, avanzamos en la dirección de máximo descenso (el opuesto al gradiente).
- El problema práctico: **¿cómo calculamos $\nabla_\theta \mathcal{L}$?**
- Respuesta: **backpropagation**.

Training the network

Backpropagation



¿Cómo funciona backpropagation?

Actualizando un peso de la capa oculta, w_{11}

- Queremos $\partial\mathcal{L}/\partial w_{11}$. Por regla de la cadena:

$$\frac{\partial\mathcal{L}}{\partial w_{11}} = \underbrace{\frac{\partial\mathcal{L}}{\partial\hat{y}}}_{-(y-\hat{y})} \cdot \underbrace{\frac{\partial\hat{y}}{\partial A_1}}_{\beta_1} \cdot \underbrace{\frac{\partial A_1}{\partial z_1}}_{\sigma'(z_1)} \cdot \underbrace{\frac{\partial z_1}{\partial w_{11}}}_{X_1}$$

- Resultando:

$$\frac{\partial\mathcal{L}}{\partial w_{11}} = -(y-\hat{y}) \cdot \beta_1 \cdot \sigma'(z_1) \cdot X_1$$

- Cada factor viene de una capa distinta: el error se *propaga hacia atrás*.

¿Cómo funciona backpropagation?

Actualizando un peso de la capa de salida, β_1

- Ahora el peso β_1 solo atraviesa la última capa:

$$\frac{\partial \mathcal{L}}{\partial \beta_1} = \underbrace{\frac{\partial \mathcal{L}}{\partial \hat{y}}}_{-(y-\hat{y})} \cdot \underbrace{\frac{\partial \hat{y}}{\partial \beta_1}}_{A_1} = -(y - \hat{y}) \cdot A_1$$

- Más corto que el de capa oculta: una sola "salto" hacia atrás.
- Observación: el factor $(y - \hat{y})$ aparece en *todas* las derivadas. Por eso se puede calcular una sola vez y reutilizar (base de backprop eficiente).

Backpropagation: resumen

- La idea completa:
  1. **Forward pass**: calcular $\hat{y}$ y $\mathcal{L}$ dado Θ actual.
  2. **Backward pass**: calcular $\nabla_\Theta \mathcal{L}$ usando la regla de la cadena de *atrás hacia adelante*, reutilizando productos intermedios.
  3. **Update**: $\Theta \leftarrow \Theta - \epsilon \nabla_\Theta \mathcal{L}$.
  4. Repetir.
- La magia es que el costo computacional de backprop es *del mismo orden* que el forward pass. (Rumelhart, Hinton & Williams, 1986.)

Agenda

1. Recap: SNN
   - Example: XOR
   - Activation Functions
   - Output Functions
2. Training the network
3. Batch, Stochastic, Mini-batch
4. Optimizadores
5. Cierre

Batch Gradient Descent

- El gradiente se calcula **usando todo el dataset** en cada paso:

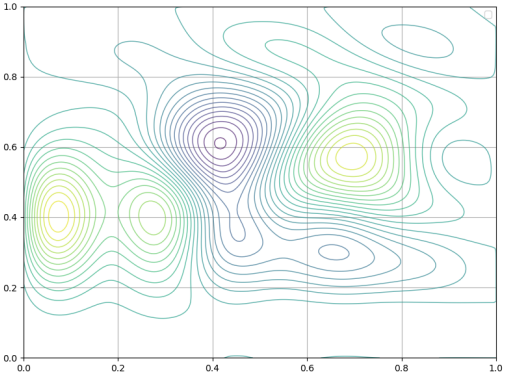
$$\theta' = \theta - \epsilon \cdot \frac{1}{n} \sum_{i=1}^{n} \nabla_\theta L(y_i, f(X_i; \theta))$$

- Trayectoria *suave*; cada paso es una estimación insesgada del gradiente verdadero.
- Terriblemente lento en datasets grandes: cada paso cuesta $O(n)$.
- Sin embargo, escala bien con el *número de variables* p: mejor que Normal Equations cuando p es enorme.

Stochastic Gradient Descent (SGD)

- En cada paso, SGD toma **una observación aleatoria** y calcula el gradiente sólo con ella:

$$\theta' = \theta - \epsilon \cdot \nabla_\theta L(y_i, f(X_i; \theta)), \quad i \sim \text{Unif}\{1, \ldots, n\}$$



- Cada paso es $O(1)$.
- Trayectoria ruidosa (zig-zag visible en las curvas de nivel).

Mini-batch Gradient Descent

- Punto medio entre Batch y SGD: en cada paso usamos un **mini-batch** de tamaño B:

$$\theta' = \theta - \epsilon \cdot \frac{1}{B} \sum_{i \in \mathcal{B}} \nabla_\theta L(y_i, f(X_i; \theta))$$

- Típicamente 2^x eg $\{32, 64, 128, 256\}$.
- **Ventajas**:
  - Compromiso: menos ruido que SGD, mucho más barato que Batch.
  - Mapea bien sobre GPUs (paralelizable dentro del batch).
  - El ruido residual ayuda a escapar de mínimos locales.

Agenda

1. Recap: SNN
   - Example: XOR
   - Activation Functions
   - Output Functions
2. Training the network
3. Batch, Stochastic, Mini-batch
4. Optimizadores
5. Cierre

¿Por qué necesitamos más que SGD?

- Problemas de SGD puro:
  - Aprende *lento* en direcciones de poca curvatura.
  - Oscila en direcciones de mucha curvatura.
  - Misma learning rate para todos los parámetros, sin importar su escala.
- Soluciones históricas, en orden:
  1. **Momentum** (Polyak, 1964): memoria del gradiente pasado.
  2. **RMSProp** (Tieleman & Hinton, 2012): learning rate adaptativa por parámetro.
  3. **Adam** (Kingma & Ba, 2014): Momentum + RMSProp.

SGD + Momentum (Polyak, 1964)

$$v' = \gamma v - \epsilon \cdot \nabla_\theta \mathcal{L}(\theta)$$
$$\theta' = \theta + v'$$

- Agrega una "inercia" al gradiente.
- Acumula dirección $\Rightarrow$ suaviza la trayectoria.
- Ayuda a escapar de mínimos poco profundos y acelera en direcciones consistentes.
- $\gamma \in [0, 1)$, típicamente 0.9.
- Lee el estado v como un *momentum físico*: el parámetro tiene inercia.

RMSProp (Tieleman & Hinton, 2012)

$$E[\nabla_\theta \mathcal{L}^2]_t = \rho\, E[\nabla_\theta \mathcal{L}^2]_{t-1} + (1-\rho)\,(\nabla_\theta \mathcal{L})^2$$

$$\theta' = \theta - \frac{\epsilon}{\sqrt{E[\nabla_\theta \mathcal{L}^2]_t + \eta}} \cdot \nabla_\theta \mathcal{L}$$

- Mantiene una *media móvil* del gradiente al cuadrado.
- Divide el gradiente por su magnitud típica reciente:
  - Parámetros con gradientes grandes $\Rightarrow$ pasos chicos.
  - Parámetros con gradientes chicos $\Rightarrow$ pasos grandes.
- **Learning rate adaptativa por parámetro**. Ya no todos avanzan al mismo ritmo.

Adam (Kingma & Ba, 2014)

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)\nabla_\theta \mathcal{L} \qquad \text{(momento 1, estilo Momentum)}$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2)(\nabla_\theta \mathcal{L})^2 \qquad \text{(momento 2, estilo RMSProp)}$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t} \qquad \text{(corrección de sesgo)}$$

$$\theta' = \theta - \epsilon \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \eta}$$

- Combina: **Momentum + RMSProp**.
- La corrección de sesgo es clave al inicio (cuando m, v arrancan en 0).
- **Default moderno** en casi todo DL aplicado: robusto, rápido, poco sensible a ϵ.

¿Cuál usar? RMSProp vs Adam vs SGD

Zhou et al. (2020)

- Aunque Adam converge **más rápido**, SGD suele encontrar soluciones que **generalizan mejor**.
- Zhou et al. (2020) modela ambos métodos teóricamente:
  - **SGD**: el ruido del gradiente estocástico actúa como regularización implícita.
    - Escapa más fácil de mínimos locales (flat vs sharp minima).
    - Alcanza soluciones con mejor generalización.
  - **Adam**: puede quedar atrapado en mínimos subóptimos (sharp minima).
- El ruido inherente al SGD actúa como una forma de regularización que favorece la generalización.
- En la práctica a veces se arranca con Adam (converge rápido), y para el *fine-tuning* final se cambia a SGD con momentum.

Agenda

1. Recap: SNN
   - Example: XOR
   - Activation Functions
   - Output Functions
2. Training the network
3. Batch, Stochastic, Mini-batch
4. Optimizadores
5. Cierre

Lo que nos llevamos de hoy

1. Una Single Layer NN es una combinación lineal seguida de una activación no-lineal.
2. La no-linealidad de la activación es la clave: es donde sucede ”la magia”
3. La activación de salida se elige según el problema (identidad / sigmoide / softmax).
4. Entrenamos con backpropagation + gradiente descendente.
5. Tres formas de estimar el gradiente: Batch, SGD, Mini-batch.
6. Optimizadores modernos: Momentum, RMSProp, Adam. El ruido del SGD ayuda a generalizar.

Next

- Teorema de aproximación universal.
- Arquitecturas profundas (multicapa).
- Regularización: dropout, early stopping.
- Aplicación clasificación multiclase: MNIST.